COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering

Full Stack Web Development

Unit III Assignment: React.js — Components, State & Hooks

Name: Suraj Singh	**CU ID:** cu240251579
Roll No.: 59	**Subject:** Full Stack Web Development (BTCS303T)

Question 1. Write a functional React component Counter using useState that increments and decrements a number on button clicks. In 1 line, explain why a plain JavaScript variable (instead of useState) would not update the UI.

Answer:

```
import { useState } from 'react';  function
Counter() {   const [count, setCount] =
useState(0);    return (
<div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(prev => prev - 1)}>Decrement</button>
<button onClick={() => setCount(prev => prev + 1)}>Increment</button>
    </div>
  );
}  export default Counter;
```

Why a plain variable fails (1 line): Changing a plain variable does not tell React to re-render (and it resets on every render), whereas `setCount` triggers a re-render with the new value.

Question 2. This component calls setCount directly inside the render body with no condition, causing an infinite re-render loop. Fix the code.

```
function Counter() {   const [count,
setCount] = useState(0);   setCount(count
+ 1);   return
<p>{count}</p>;
}
```

Answer:

Problem: Calling `setCount` during render schedules another render, which calls `setCount` again, and so on forever (React throws *Too many re-renders*). State updates must happen in event handlers or effects, never directly in the render body.

Fixed code (update state from an event handler):

```
import { useState } from 'react';  function
Counter() {   const [count, setCount] =
useState(0);    return (
<div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  ); }
```

If the goal was to increment automatically **once** when the component loads, use an effect with an empty dependency array instead:

```
useEffect(() => {   setCount(prev => prev + 1);   // runs only once,
after the first render }, []);
```

Question 3. Write a useEffect hook that fetches data from /api/posts when the component first mounts. In 1 line, explain why the empty dependency array [] is necessary here.

Answer:

```
import { useState, useEffect } from 'react';
function Posts() {   const [posts, setPosts] =
useState([]);   const [loading, setLoading] =
useState(true);   const [error, setError] =
useState(null);    useEffect(() => {     async
function fetchPosts() {       try {         const
res = await fetch('/api/posts');
        if (!res.ok) throw new Error(`HTTP error ${res.status}`);
const data = await res.json();         setPosts(data);       } catch (err) {
setError(err.message);       } finally {         setLoading(false);       }
}     fetchPosts();   }, []);          // [] -> run only once, right after the
first render (mount)    if (loading) return <p>Loading...</p>;   if (error)
return <p>Error: {error}</p>;    return (     <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  ); }
```

Why [] is needed (1 line): Without it the effect runs after every render, and since `setPosts` causes a rerender, it would fetch again and again in an infinite loop.

Question 4. In this code, two sibling components each keep their own local copy of the same value, so they fall out of sync. Refactor the code to lift the shared state up to the parent component.

```
function PriceInput() {   const [price, setPrice] = useState(0);   return
<input value={price} onChange={e => setPrice(e.target.value)} />; }
function PriceDisplay() {   const [price, setPrice] = useState(0);   return
<p>Price: {price}</p>;
}
```

Answer:

Idea: Keep **one** copy of the state in the closest common parent and pass it down as props (the value to both, and the setter to the component that changes it).

Refactored code:

```
import { useState } from 'react';  function PriceParent() {   const [price,
setPrice] = useState(0);          // single source of truth    return (
<div>
      <PriceInput price={price} onPriceChange={setPrice} />
      <PriceDisplay price={price} />
    </div>
  );
}  function PriceInput({ price, onPriceChange }) {   return
(     <input       type="number"       value={price}
onChange={e => onPriceChange(e.target.value)}     />
  );
}  function PriceDisplay({ price }) {   return <p>Price: {price}</p>;
// always in sync with the input
}
```

Question 5. Write a custom React hook useLocalStorage(key, initialValue) that behaves like useState but also saves and reads the value from localStorage.

Answer:

Custom hook (useLocalStorage.js):

```
import { useState, useEffect } from 'react';  function
useLocalStorage(key, initialValue) {   // Lazy initializer: reads
localStorage only on the first render   const [value, setValue] =
useState(() => {     try {       const stored =
localStorage.getItem(key);       return stored !== null ?
JSON.parse(stored) : initialValue;     } catch (error) {
console.error('Error reading localStorage', error);       return
initialValue;     }
  });
   //   Save   whenever   the   key   or   value   changes
useEffect(() => {    try {     localStorage.setItem(key,
JSON.stringify(value));           }   catch   (error)   {
console.error('Error writing localStorage', error);    }
  }, [key, value]);    return [value, setValue];      // same
shape as useState }  export default useLocalStorage;
```

Usage:

```
function ThemeToggle() {   const [theme, setTheme] =
useLocalStorage('theme', 'light');    return (     <button onClick={() =>
setTheme(theme === 'light' ? 'dark' : 'light')}>
      Current theme: {theme}
    </button>
  );
}
```

Question 6. This list renders correctly, but React logs a "key" warning and re-renders inefficiently. Fix the code.

```
{items.map(item => (
  <li>{item.name}</li>
))}
```

Answer:

Fixed code:

```
{items.map(item => (
  <li key={item.id}>{item.name}</li> ))}
```

Why: React uses the `key` prop to identify each list item between renders. Without a stable, unique key it cannot tell which items were added, removed or moved, so it re-creates or re-renders more elements than needed and can mix up component state.

Use a unique id from the data (like `item.id`). Avoid the array index as a key when the list can be reordered, filtered or have items inserted or deleted.

Question 7. Write a reusable React <Button> component that accepts a variant prop ("primary" or "danger") and applies a different CSS class depending on its value.

Answer:

Button.js:

```
import './Button.css';  function Button({ variant = 'primary', children,
onClick, type = 'button' }) {   return (     <button type={type} className={`btn
btn-${variant}`} onClick={onClick}>
      {children}
    </button>
  );
}  export default Button;
```

Button.css:

```
.btn {   padding:
10px 20px;   border:
none;   borderradius:
6px;   fontsize:
1rem;   cursor:
pointer; }
.btn-primary { background: #0d6efd; color: #ffffff; }
.btn-danger  { background: #dc3545; color: #ffffff; }
```

Usage:

```
<Button variant="primary" onClick={handleSave}>Save</Button>
<Button variant="danger" onClick={handleDelete}>Delete</Button>
```

Question 8. In this app, ChildCard re-renders every time ParentDashboard re-renders, even when its own props haven't changed. Explain briefly why this happens, then fix it using React.memo.

```
function ParentDashboard() {   const
[tick, setTick] = useState(0);
return (     <div>
      <button onClick={() => setTick(tick + 1)}>Refresh</button>
      <ChildCard title="Static Title" />
    </div>
  ); } function ChildCard({ title
}) {   console.log('ChildCard
rendered');   return
<h3>{title}</h3>;
}
```

Answer:

Why it happens: By default, when a parent component re-renders (here because `tick` changed), React rerenders **all** of its children too, without checking whether their props actually changed.

Fixed code (wrap the child in `React.memo`):

```
import { useState, memo } from
'react';  function ParentDashboard() {
const [tick, setTick] = useState(0);   return
(     <div>
      <button onClick={() => setTick(tick + 1)}>Refresh</button>
      <ChildCard title="Static Title" />
    </div>
  );
}  const ChildCard = memo(function ChildCard({ title }) {
console.log('ChildCard rendered');       // now logs only on the first render
return <h3>{title}</h3>;
});
```

Note: `memo` does a shallow comparison of props, so it works here because `title` is a string. If you pass objects or functions as props, stabilize them with `useMemo` / `useCallback`.

Question 9. Write React Router (v6) code to set up 3 routes - "/", "/products", and "/products/:id" - including a working <Link> for each in the navbar.

Answer:

Install: `npm install react-router-dom`
App.js:

```
import { BrowserRouter, Routes, Route, Link, useParams } from 'react-routerdom';
function Home() {   return <h1>Home Page</h1>; }  function Products() {   return
<h1>All Products</h1>; }  function ProductDetail() {   const { id } =
useParams();                  // reads :id from the URL   return <h1>Product
Details - ID: {id}</h1>; }  function Navbar() {   return (     <nav style={{
display: 'flex', gap: '16px' }}>
      <Link to="/">Home</Link>
      <Link to="/products">Products</Link>
      <Link to="/products/101">Product 101</Link>
    </nav>
  );
}  function App()
{   return (
<BrowserRouter>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<Products />} />
        <Route path="/products/:id" element={<ProductDetail />} />
      </Routes>
    </BrowserRouter>
  );
}  export default App;
```

Note: `<Link>` changes the URL without a full page reload, and `useParams()` gives the value of `:id` (for example 101) inside `ProductDetail`.

Question 10. This form has a separate useState for every field, making the code repetitive. Refactor it into a single state object with one shared onChange handler.

```
const [name, setName] = useState(''); const
[email, setEmail] = useState(''); const
[phone, setPhone] = useState('');
```

Answer:

```
import { useState } from 'react';  function
ContactForm() {   const [formData, setFormData]
= useState({     name: '',     email: '',
phone: ''   });
   // ONE handler for every input   const handleChange
= (e) => {     const { name, value } =
e.target;     setFormData(prev => ({ ...prev, [name]: value
}));   };    const handleSubmit = (e) => {
    e.preventDefault();     console.log(formData);        // { name: '...',
email: '...', phone: '...' }   };    return (     <form
onSubmit={handleSubmit}>
      <input name="name"  value={formData.name}  onChange={handleChange} placeholder="Name" />
 <input name="email" value={formData.email} onChange={handleChange} placeholder="Email" />
 <input name="phone" value={formData.phone} onChange={handleChange} placeholder="Phone" />
      <button type="submit">Submit</button>
    </form>
  ); }
```

How it works: each input's `name` attribute matches a key in the state object. The handler uses the computed property `[name]: value` to update only that field, and the spread `...prev` keeps the other fields unchanged.